

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1711

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour
Total Marks:50

Design Task

Code of Conduct:

*I **B.Bhargavi 192311136** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B.Bhargavi

Design Task

1. **Hybrid AI Recommendation Engine Design**
Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. **Smart Disaster Detection System**
Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework**
Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. **Autonomous Supply Chain Optimization System**
Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. **Personalized Learning Analytics Platform**
Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration

Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

1. Hybrid AI Recommendation Engine Design

Introduction

A Hybrid AI Recommendation Engine combines Content-Based Filtering and Collaborative Filtering to provide accurate and personalized recommendations. PySpark RDDs are used to process large amounts of user-item interaction data in a distributed environment.

For example, an online shopping system can recommend products by considering both the user's previous purchases and the preferences of similar users.

System Architecture

The major components are:

1. Data Collection

- Collect user information, product information, ratings, clicks, purchases, and search history.
- Data can come from websites, mobile applications, and transaction databases.

2. Data Storage

- Store large datasets using HDFS or cloud storage.

- Data is divided into multiple partitions.
- 3. PySpark Processing Layer
 - Load the data into PySpark RDDs.
 - RDDs allow distributed processing across multiple machines.
- 4. Content-Based Filtering
 - Recommends items similar to those previously liked by the user.
 - Item features such as category, price, brand, and description are compared.
- 5. Collaborative Filtering
 - Finds users with similar behavior.
 - Recommendations are generated from items liked or purchased by similar users.
- 6. Hybrid Recommendation
 - Combines both recommendation scores.
 - For example:

Final Score = α (Content Score) + β (Collaborative Score)

- 7. Intelligent Agent
 - Observes user behavior.
 - Updates recommendations when the user provides feedback.

Use of PySpark RDDs

RDDs support operations such as:

- `map()` – transforms each record.
- `filter()` – removes unwanted records.
- `flatMap()` – creates multiple records from one record.
- `reduceByKey()` – combines values having the same key.
- `join()` – combines related datasets.
- `groupByKey()` – groups data based on keys.

For example, millions of ratings can be distributed across different worker nodes. Each node processes its partition, and the results are combined to generate recommendations.

Role of Intelligent Agents

Intelligent agents continuously monitor:

- User clicks
- Search history
- Ratings
- Purchases
- Time spent on products
- Likes/dislikes
- Feedback

The agent changes recommendations according to these behaviors.

For example, if a user repeatedly searches for laptops, the agent increases the priority of laptops and related accessories.

Advantages

- Handles millions of users and products.
- Provides personalized recommendations.
- Distributed processing reduces processing time.
- Combines two recommendation approaches.
- Agents allow real-time adaptation.
- Improves user experience and customer satisfaction.

Conclusion

A hybrid recommendation engine using PySpark RDDs provides scalable and personalized recommendations. Content-based filtering understands item similarity, while collaborative filtering learns from user behavior. Intelligent agents continuously adapt the system based on feedback.

2. Smart Disaster Detection System

Introduction

A Smart Disaster Detection System uses AI and distributed computing to identify natural disasters such as floods, earthquakes, landslides, and cyclones at an early stage.

The system collects information from multiple sources such as satellite images, IoT sensors, weather stations, and social media. PySpark RDDs process this large amount of data in parallel.

System Architecture

The system consists of:

Data Sources → Data Collection → PySpark RDD Processing → Data Fusion → AI Detection → Intelligent Agents → Early Warning

1. Data Collection

Data is collected from:

- Satellite images
- Water-level sensors
- Seismic sensors
- Weather stations
- GPS devices
- Social media posts
- Emergency reports

2. Distributed Data Processing

PySpark RDDs divide large datasets into partitions.

For example, satellite images from different geographical regions can be distributed across multiple worker nodes.

RDD operations such as:

- `map()`
- `filter()`
- `reduceByKey()`
- `groupByKey()`

can process the information in parallel.

3. Data Cleaning

The system removes:

- Duplicate data

- Missing values
- Incorrect sensor readings
- Irrelevant social media posts

This improves the quality of disaster detection.

4. Data Fusion

Data fusion means combining information from different sources.

For example:

Heavy rainfall + Rising river level + Satellite water coverage = High probability of flood

Similarly:

Seismic sensor activity + Multiple nearby sensor alerts = Possible earthquake

Combining multiple sources makes the prediction more reliable.

5. AI-Based Detection

AI models analyze the processed data to identify disaster patterns.

The system can classify conditions as:

- Normal
- Warning
- High Risk
- Emergency

6. Intelligent Agents

Different intelligent agents can perform different tasks.

Sensor Agent

- Monitors sensor values.

Satellite Agent

- Analyzes satellite information.

Social Media Agent

- Detects disaster-related posts.

Decision Agent

- Combines information and estimates disaster severity.

Alert Agent

- Sends warnings to authorities and affected people.

7. Early Warning

When a high-risk condition is detected, the system can generate:

- Mobile alerts
- Emergency notifications
- Evacuation recommendations
- Location-based warnings
- Rescue team notifications

Advantages

- Early disaster detection.
- Handles large-scale data.
- Combines multiple information sources.
- Faster processing using distributed computing.
- Reduces false alarms through data fusion.
- Supports emergency response.

Conclusion

The Smart Disaster Detection System combines PySpark, AI, multi-source data fusion, and intelligent agents. Distributed processing allows large amounts of sensor, satellite, and social media data to be analyzed quickly, enabling early warnings and better disaster management.

3. AI-Based Fraud Detection Framework

Introduction

Financial institutions process millions of transactions every day. Some transactions may be fraudulent. An AI-based Fraud Detection Framework uses PySpark RDDs and anomaly detection techniques to identify suspicious transactions efficiently.

System Architecture

Transaction Data → PySpark RDD → Data Processing → Feature Extraction → Anomaly Detection → Intelligent Agents → Alert

1. Transaction Data Collection

The system collects:

- Transaction amount
- Transaction time
- Account information
- Location
- Device information
- Transaction frequency
- Previous transaction history

2. RDD-Based Distributed Processing

Large transaction datasets are divided into partitions.

For example, one million transactions can be distributed among several worker nodes.

RDD operations can be used to:

- Remove invalid transactions using `filter()`.
- Extract features using `map()`.
- Group transactions using `groupByKey()`.
- Calculate statistics using `reduceByKey()`.

3. Feature Extraction

Important features include:

- Average transaction amount
- Number of transactions per hour
- Location changes
- Unusual spending patterns
- Number of failed transactions
- Transaction frequency

4. Anomaly Detection

Anomaly detection identifies transactions that are significantly different from normal behavior.

For example:

A customer normally spends ₹1,000–₹5,000 per transaction. Suddenly, a transaction of ₹2,00,000 occurs from a new location. The system can classify it as suspicious.

Possible techniques include:

- Statistical anomaly detection
- Clustering
- Isolation-based methods
- Rule-based detection

5. Intelligent Agent Collaboration

Different agents work together.

Transaction Monitoring Agent

- Monitors transactions.

User Behavior Agent

- Learns normal customer behavior.

Anomaly Agent

- Detects unusual transactions.

Risk Agent

- Calculates fraud risk.

Alert Agent

- Sends alerts when risk exceeds a threshold.

6. Continuous Learning

The system receives feedback about whether an alert was:

- Genuine fraud
- False alarm

This feedback is used to improve future detection.

Example

Suppose a user normally logs in from Chennai and makes small transactions.

Suddenly:

1. Login occurs from another country.
2. Large transaction is attempted.
3. Several transactions occur within a few minutes.

The intelligent agents identify these unusual patterns and increase the fraud score.

Advantages

- Processes millions of transactions.
- Faster fraud detection.
- Scalable architecture.
- Reduces financial losses.
- Supports real-time alerts.
- Continuously improves using feedback.

Conclusion

PySpark RDDs provide scalable distributed transaction processing, while anomaly detection identifies unusual financial behavior. Intelligent agents collaborate to monitor transactions, calculate risk, generate alerts, and continuously improve the fraud detection system.

4. Autonomous Supply Chain Optimization System

Introduction

Supply chain management involves inventory, transportation, warehouses, suppliers, and customer demand. An Autonomous Supply Chain Optimization System uses PySpark and intelligent agents to make decentralized decisions and improve efficiency.

System Architecture

Supplier Data + Inventory Data + Transportation Data + Sales Data → PySpark → Forecasting → Optimization → Intelligent Agents → Decisions

1. Data Collection

The system collects:

- Sales data
- Inventory levels
- Supplier information
- Transportation costs
- Delivery times
- Customer demand
- Warehouse information

2. RDD Processing

Large datasets are divided into partitions and processed across multiple nodes.

Important transformations include:

`map()`

- Converts raw data into useful formats.

`filter()`

- Removes incorrect or unnecessary records.

`reduceByKey()`

- Calculates total sales or inventory.

`groupByKey()`

- Groups information by product or location.

`join()`

- Combines sales, inventory, and product information.

3. Demand Forecasting

Historical sales data is analyzed to estimate future demand.

For example:

If the demand for a product increases every weekend, the system can predict increased weekend demand and maintain sufficient inventory.

4. Inventory Optimization

The system calculates:

- Current stock
- Expected demand
- Reorder level
- Safety stock
- Storage cost

When inventory falls below the required level, the system can recommend or initiate a reorder.

5. Logistics Optimization

The system considers:

- Distance
- Fuel cost
- Delivery time
- Vehicle availability
- Traffic conditions

It selects efficient transportation routes.

6. Intelligent Agents

Multiple agents cooperate.

Inventory Agent

- Monitors stock levels.

Demand Agent

- Predicts customer demand.

Supplier Agent

- Selects suitable suppliers.

Logistics Agent

- Plans transportation.

Warehouse Agent

- Manages warehouse operations.

Decision Agent

- Coordinates the overall system.

7. Decentralized Decision Making

Agents can make local decisions while communicating with other agents.

For example:

If the inventory agent detects low stock, it communicates with the supplier agent. The supplier agent checks availability and cost. The logistics agent then selects an efficient delivery method.

Advantages

- Reduces inventory costs.
- Improves delivery speed.
- Prevents stock-outs.
- Reduces transportation costs.
- Handles large datasets.
- Supports autonomous decision-making.
- Improves overall supply chain efficiency.

Conclusion

The Autonomous Supply Chain Optimization System uses PySpark RDDs for large-scale processing and intelligent agents for decentralized decision-making. By combining demand forecasting, inventory management, and logistics optimization, the system can reduce costs and improve supply chain performance.

5. Personalized Learning Analytics Platform

Introduction

A Personalized Learning Analytics Platform uses AI and PySpark to analyze student behavior and performance. It provides personalized learning materials and recommendations based on each student's strengths, weaknesses, and learning progress.

System Architecture

Student Data → PySpark RDD → Data Processing → Learning Analytics → Student Profiling → Intelligent Agent → Personalized Content

1. Data Collection

The platform collects:

- Student marks
- Quiz results
- Assignment scores
- Attendance
- Video watching time
- Course completion
- Number of attempts
- Learning speed
- Interaction with learning materials

2. PySpark RDD Processing

Student data is distributed across multiple worker nodes.

RDD operations can be used to:

- `map()` – transform student records.
- `filter()` – remove incomplete records.
- `reduceByKey()` – calculate average scores.
- `groupByKey()` – group data by student or course.
- `join()` – combine student performance with course information.

3. Student Profiling

The system creates a learning profile for each student.

For example:

Student A

- Mathematics: Strong
- Programming: Medium

- Database: Weak

The platform can recommend more database learning materials to Student A.

4. Adaptive Learning

The system changes learning content based on student performance.

If a student performs poorly:

- Provide easier explanations.
- Recommend additional examples.
- Give practice questions.
- Provide revision materials.

If a student performs well:

- Provide advanced topics.
- Give challenging questions.
- Recommend additional projects.

5. Intelligent Agents

Different agents can perform different tasks.

Student Monitoring Agent

- Tracks learning activities.

Performance Agent

- Analyzes marks and quiz results.

Content Agent

- Selects suitable learning materials.

Feedback Agent

- Provides personalized feedback.

Recommendation Agent

- Recommends the next learning activity.

6. Real-Time Feedback

When a student completes a quiz, the system immediately analyzes the result.

For example:

Score < 40% → Revision required

Score 40–70% → More practice

Score > 70% → Advanced content

This allows the platform to adapt continuously.

7. Scalability

PySpark allows thousands or millions of student records to be processed in parallel.

For example:

- Node 1 → Student data 1–10,000
- Node 2 → Student data 10,001–20,000
- Node 3 → Student data 20,001–30,000

The results are combined to generate large-scale learning analytics.

Advantages

- Personalized learning.
- Real-time feedback.
- Supports thousands of learners.
- Identifies weak subjects.
- Recommends suitable content.
- Reduces manual monitoring by teachers.
- Improves student performance.

Conclusion

The Personalized Learning Analytics Platform combines PySpark RDDs, AI analytics, and intelligent agents to understand student behavior. It continuously adapts learning content based on performance and feedback, providing scalable and personalized education.

Common Architecture for All Five Systems

A simple architecture that can be remembered for the exam is:

Data Sources → Distributed Storage → PySpark RDD Processing → AI/ML Analysis → Intelligent Agents → Decision/Recommendation → Feedback → Continuous Improvement

Key role of PySpark RDD

RDD = Resilient Distributed Dataset

It allows large datasets to be:

- Distributed across multiple machines.
- Processed in parallel.
- Transformed using operations such as `map()`, `filter()`, and `reduceByKey()`.
- Recovered from failures because RDDs are fault tolerant.

Key role of Intelligent Agents

Intelligent agents:

1. Observe the environment.
2. Collect new information.
3. Analyze the current state.
4. Make decisions.
5. Take appropriate actions.
6. Receive feedback.
7. Adapt future decisions.